**HCLTech Career Shaper Programme**

III Year Industry Project · Project Report

# Early Disease Prediction Using Machine Learning on Health Data

*Three models, SHAP explanations and a Streamlit prototype on a partly synthetic clinical dataset*

| | |
|---|---|
| **Project number** | 29 |
| **Domain** | Digital-Data-AI / AIML |
| **Difficulty / duration** | Advanced · 4 weeks · individual project |
| **Academic year** | 2025-26 |
| **Submitted by** | [Your full name] |
| **Roll number / learner ID** | [Your roll number] |
| **College** | [Your college name] |
| **Mentor** | [Mentor name] |
| **Submission date** | [Date] |

**Disclaimer**

This project is a student research prototype built on partly synthetic data. It must **not** be used for clinical diagnosis or medical decisions without supervision by qualified medical professionals.

# Contents

1 Introduction
2 Dataset
3 Methodology
4 Exploratory data analysis
5 Results
6 Explainability with SHAP
7 The Streamlit application
8 Testing and reproducibility
9 Ethical considerations
10 Limitations
11 Conclusion and future work
References
Appendix A Project structure and how to run
Appendix B Environment

# Abstract

Early detection of common diseases such as diabetes, heart disease and high blood pressure gives people more time to change their habits and see a doctor. This project builds a machine learning pipeline that estimates the chance of disease from six routine measurements: age, BMI, blood pressure, glucose, cholesterol and smoking status. The 500 patient records provided by the programme were extended with 4,500 generated records, giving 5,000 in total.

Three models (Logistic Regression, Random Forest and XGBoost) were tuned with 5-fold cross-validation on 80% of the data and tested once on the remaining 20%. They performed almost the same, with a test ROC-AUC of about 0.82. Logistic Regression had the best cross-validated score and was selected (test ROC-AUC 0.822, 95% interval 0.796 to 0.848). SHAP explanations show that glucose is by far the strongest driver, followed by blood pressure, age, BMI, cholesterol and smoking.

An important finding is that the 500 original records contain no learnable signal (cross-validated AUC of about 0.5). The headline result therefore comes from relationships that were built into the generated rows. It shows that the pipeline works, not that the model would work on real patients. A Streamlit app, three Jupyter notebooks and 34 automated tests complete the prototype. The model is for learning only and must not be used for diagnosis.

# 1 Introduction

## 1.1 Background

Many long-term health problems, such as diabetes, heart disease and high blood pressure, build up slowly. In the early stage a person may feel fine, yet routine measurements like blood glucose, blood pressure, body weight and cholesterol can already show warning signs. If those signs are noticed early, there is more time to change habits and to see a doctor.

Machine learning can combine several of these measurements into one risk estimate. A tool like this does not replace a doctor. It only helps decide who should get a closer look first.

## 1.2 Problem statement

Given six routine measurements of a patient (age, BMI, blood pressure, glucose, cholesterol and smoking status), estimate the probability that the patient has the disease, and explain which measurements drove the estimate.

## 1.3 Objectives

1. Prepare, validate and document the dataset, and extend it from 500 to 5,000 rows.
2. Explore the data with at least three visualisations (the EDA notebook has six, and this report shows five of them).
3. Train and compare Logistic Regression, Random Forest and XGBoost with a fair, leak-free evaluation.
4. Handle class imbalance carefully and report metrics that go beyond accuracy.
5. Explain the predictions with SHAP, for the whole model and for single patients.
6. Build a working prototype (a Streamlit app) that shows the estimate, its explanation and a clear disclaimer.
7. Document the ethical points and the limits of the work honestly.

## 1.4 Scope

The project brief names diabetes, heart disease and hypertension as target conditions. The dataset, however, has **one combined yes/no label** and does not say which disease it stands for. The models therefore predict "disease present" and not a specific condition.

# 2 Dataset

## 2.1 Provided data

The programme folder contains `dataset.csv` with 500 patients and 8 columns: an anonymous patient id, six clinical features and a disease label. The dataset-generation prompt in the brief mentions the Pima Indians Diabetes dataset [8], and the reference list of the brief gives a UCI link [1]. That link opens the UCI “Diabetes” dataset (AIM ’94), which holds time-stamped insulin and glucose logs and is a different dataset. Neither matches the columns of the provided file, and the exact origin of the file is not stated, so it is treated simply as the dataset supplied by the programme. The mentor is asked to confirm the correct reference.

## 2.2 Extension to 5,000 rows

The dataset-generation prompt in the brief asks for the file to be extended to 5,000 rows. The 4,500 new rows (ids P0501 to P5000) were produced by the script `src/generate_synthetic_rows.py`, with a fixed random seed so the result can be reproduced exactly. The script works as follows:

- **Features** are drawn evenly inside the ranges seen in the original file: age 25 to 75, BMI 18.0 to 39.8, blood pressure 60 to 140, glucose 70 to 200, cholesterol 150 to 280. Smoking is 0 or 1 with equal chance. This keeps the new rows looking like the old ones.
- **Labels** come from an *assumed* risk score. Older age, higher BMI, blood pressure, glucose and cholesterol, and smoking all raise the chance of disease. Glucose has the strongest effect. Extra steps are added at glucose 126 or more, blood pressure 130 or more and BMI 30 or more. Random noise is added so the link is not perfect.
- **Balance.** The number of disease labels is fixed so that the full 5,000-row file has exactly 2,500 disease and 2,500 no-disease patients. The original 500 rows have 39.4% disease.

> **Important**
> The relationships in the 4,500 generated rows are **assumptions made by the generator**. They are not findings about real patients. Throughout this report the data is called *partly synthetic*, and results are interpreted with that in mind.

## 2.3 Data dictionary

| Column | Type | Meaning | Range |
|---|---|---|---|
| `patient_id` | text | Anonymous patient code. P0001 to P0500 are original, P0501 to P5000 are generated. | P0001 to P5000 |
| `age` | integer | Age in years | 25 to 75 |
| `bmi` | float | Body Mass Index in kg/m² (1 decimal) | 18.0 to 39.8 |
| `blood_pressure` | integer | Blood pressure in mmHg. One value per patient. The source does not say if it is systolic or diastolic. | 60 to 140 |
| `glucose` | integer | Blood glucose in mg/dL (unit inferred from the range). Fasting or random is not stated. | 70 to 200 |
| `cholesterol` | integer | Total cholesterol in mg/dL (unit inferred from the range) | 150 to 280 |
| `smoking` | 0 / 1 | 1 = smoker, 0 = non-smoker | 0 or 1 |
| `label` | 0 / 1 | Target. 1 = disease present, 0 = no disease (one combined label) | 0 or 1 |

## 2.4 Data quality

A validation function (`validate_data` in `src/data_utils.py`) checks every file before it is used. Cleaning removes incomplete, duplicate or out-of-range rows. It does not clip values, so no invented numbers enter the data. On the shipped file nothing needed to be removed.

| Check | Result |
|---|---|
| Rows and columns | 5,000 rows, 8 columns, in the expected order |
| Missing values | 0 |

| Check | Result |
|---|---|
| Duplicate patient ids / duplicate records | 0 / 0 |
| Values outside the allowed range | 0 in all six features |
| Label values | Only 0 and 1 |
| Class balance | 2,500 disease and 2,500 no disease (original rows alone: 39.4% disease) |

# 3 Methodology

## 3.1 Overall pipeline

The work follows seven steps, shown below. Every step is written as reusable code in the `src` package, and the notebooks call the same functions, so the report, the notebooks and the app always agree.

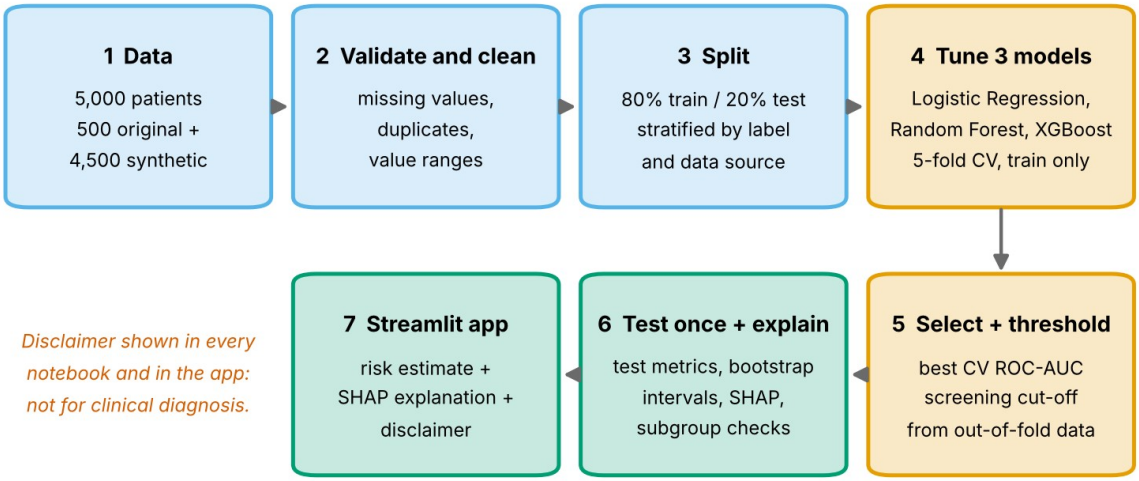



***Figure 1.*** *Project pipeline from data to app.*

## 3.2 Splitting and cross-validation

The data was split once into 80% training rows (4,000) and 20% test rows (1,000). The split is **stratified by label and by data source**, so both parts hold 50% disease cases and 10% original rows. All tuning used 5-fold cross-validation on the training rows only. The test rows were used once, at the end. The final model was chosen by cross-validated ROC-AUC, not by the test score, so the test set stays an honest check.

Scaling is part of the scikit-learn pipeline. It is therefore learned from the training folds only, which prevents information from validation or test rows leaking into training (data leakage). A unit test checks this.

## 3.3 Models and tuning

Three models were compared. Logistic Regression is simple and easy to explain. Random Forest [4] lets many decision trees vote and can capture bends and interactions. XGBoost [3] builds trees one after another, each correcting the errors of the earlier ones. Logistic Regression was searched over a full grid

of 9 values for its regularisation strength C. Random Forest and XGBoost each used 20 random draws from the search space.

| Model | Main settings searched | Selected by cross-validation |
|---|---|---|
| Logistic Regression | C from 0.01 to 100 (9 values) | C = 0.03 |
| Random Forest | trees 200 to 500, depth 3 to 12, min leaf size 3 to 40, features per split | 500 trees, depth 6, min leaf 3, features: sqrt |
| XGBoost | trees 100 to 400, depth 2 to 5, learning rate 0.01 to 0.1, sampling, regularisation | 400 trees, depth 2, learning rate 0.03, subsample 0.85, colsample 0.7, lambda 10 |

## 3.4 Handling class imbalance

The brief asks to "handle class imbalance carefully". The original file is mildly imbalanced (39.4% disease) and the extended file is balanced, so the pipeline is built to be safe in both cases:

- The split is stratified, so train and test keep the same class ratio.
- Models use class weights (`balanced` for Logistic Regression, `balanced_subsample` for Random Forest, `scale_pos_weight` for XGBoost). On the balanced file these weights are about 1 and change little. They stay in the code so it keeps working if a more imbalanced file is used.
- Rows are never copied or invented to balance the data, and the test set is never resampled.
- Results are reported with recall, specificity, precision, F1 and PR-AUC, and not with accuracy alone. A majority-class baseline gets an accuracy of 0.50 and an AUC of 0.50.

## 3.5 Evaluation metrics

| Metric | Plain meaning |
|---|---|
| Accuracy | Share of all patients classified correctly. |
| Precision | Of the patients flagged as sick, the share who really are sick. |
| Recall (sensitivity) | Of the patients who really are sick, the share the model catches. Most important for an early-warning tool. |
| Specificity | Of the healthy patients, the share correctly left unflagged. |
| F1 score | One number that balances precision and recall. |
| ROC-AUC | Chance that a random sick patient gets a higher score than a random healthy one. 0.5 is guessing, 1.0 is perfect. |
| PR-AUC | Area under the precision-recall curve. Useful when the sick cases matter most. |
| Brier score | Average squared error of the predicted probabilities. Lower is better. |

## 3.6 Screening threshold

A model gives a probability. Turning it into a yes/no flag needs a cut-off. The usual 0.50 treats both mistakes as equally bad. For early detection, missing a sick patient is worse than flagging a healthy one. A **screening threshold** was therefore chosen: the highest cut-off that still catches at least 85% of sick patients. It was found from out-of-fold predictions on the **training** data only, giving 0.39. The test set had no say in this choice.

### 3.7 Explainability

SHAP [5] shares out each prediction among the features. Positive values push the risk up and negative values push it down. The notebooks use the standard fast explainer for each model type (Linear for Logistic Regression, Tree for Random Forest and XGBoost). The app uses an exact calculation in **probability units**, so a single result reads as "+16 percentage points of risk".

## 4 Exploratory data analysis

The full analysis is in `notebooks/01_EDA.ipynb`. The main points are below.

### 4.1 Class balance

The original rows have 39.4% disease cases, the generated rows 51.2%, and the full file exactly 50%.

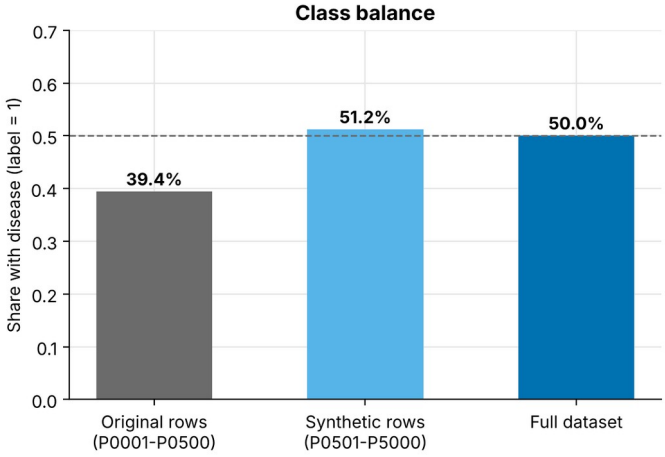



***Figure 2.*** *Share of disease cases in the original rows, the generated rows and the full file.*

### 4.2 Distributions

Every feature is spread fairly evenly across its range, with no strong outliers. This is typical of generated data. Real clinical data is usually bell-shaped with a long tail. In the generated rows the groups separate most clearly on glucose.

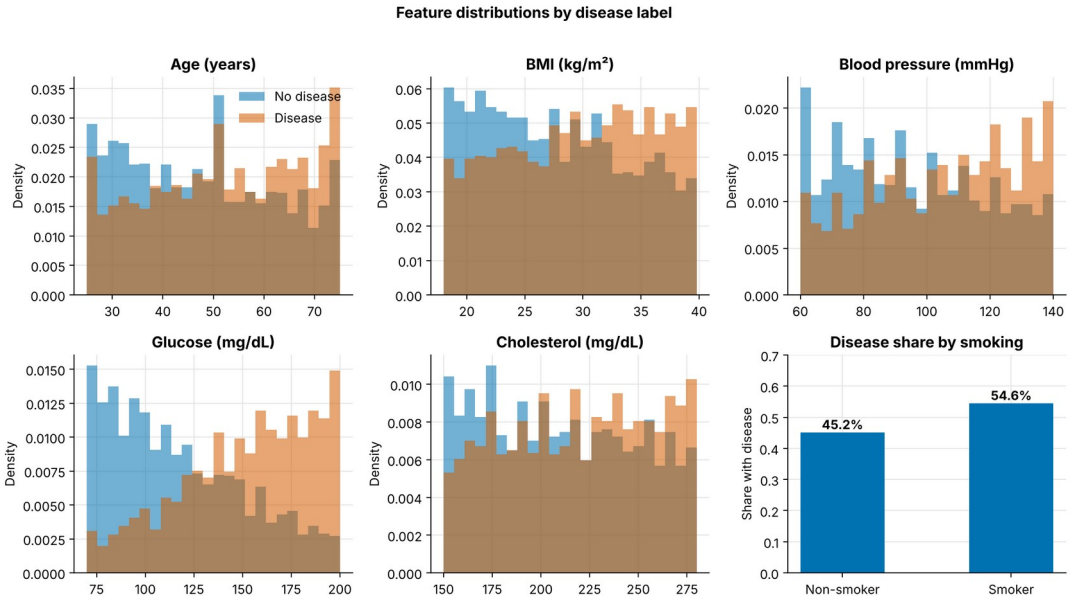



*Figure 3. Feature distributions for patients with and without disease.*

## 4.3 Correlation with the label

On the full data, glucose has the strongest link to the label (correlation 0.44), followed by blood pressure (0.19), age (0.14), BMI (0.14), cholesterol (0.11) and smoking (0.09). The features are almost uncorrelated with each other, so each adds its own information.

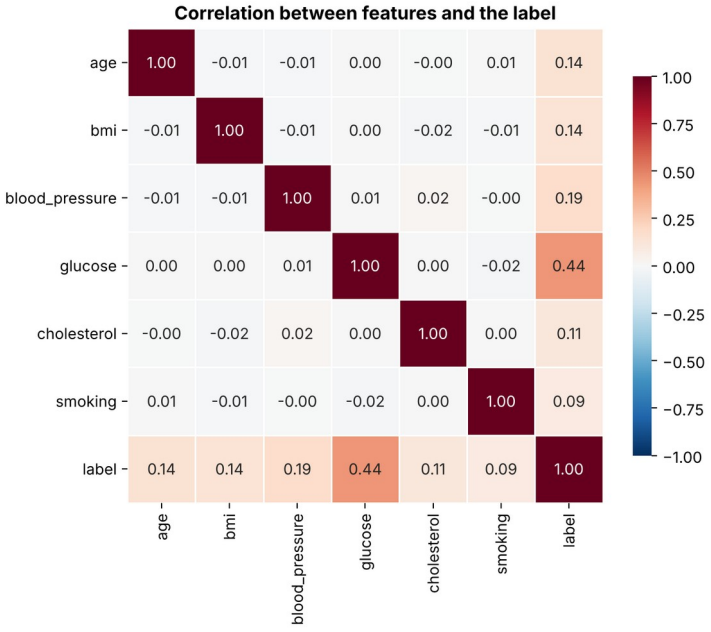



*Figure 4. Correlation matrix of the features and the label (full data).*

## 4.4 Disease share by clinical band

In the generated rows the disease share rises from 18% for glucose below 100, to 35% for 100 to 125, to 70% for 126 or higher. Blood pressure, age, BMI, cholesterol and smoking show smaller steps in the same direction. In the original rows (grey bars) the shares stay between 36% and 41% for all glucose bands. Blood pressure is split into quartiles because the file does not say if it is systolic or diastolic.

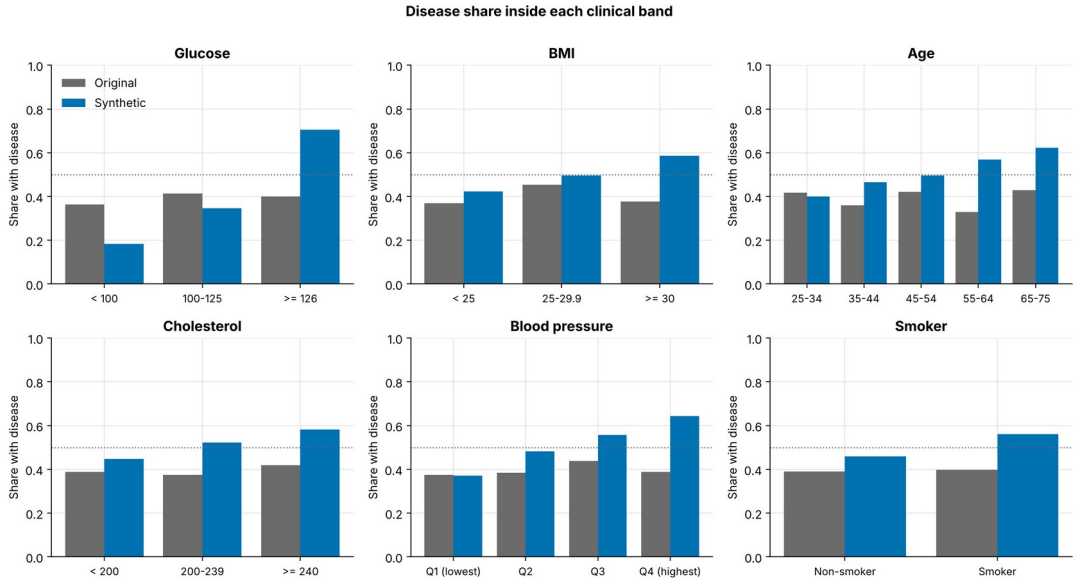



*Figure 5. Disease share inside clinical bands, original rows versus generated rows.*

## 4.5 Original rows versus generated rows

The same analysis was repeated separately for the two parts of the file. A single-feature AUC of 0.5 means a feature cannot separate the classes.

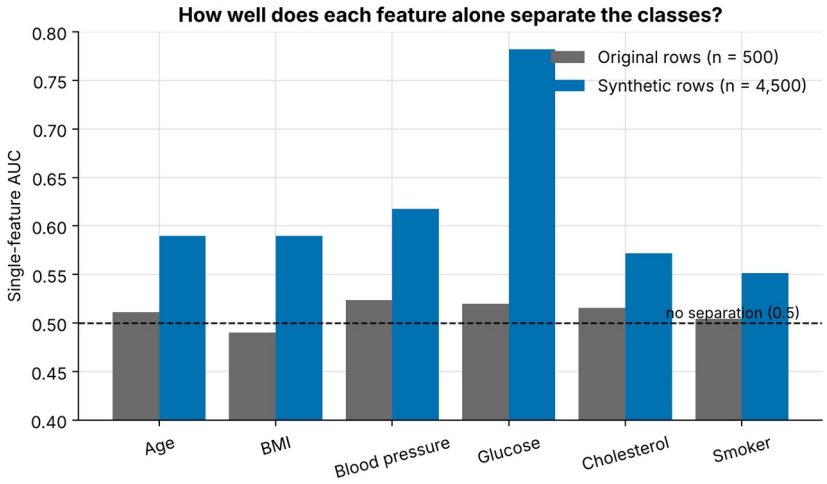



*Figure 6. Single-feature AUC for the 500 original rows and the 4,500 generated rows.*

**Finding**

In the 500 original rows **no feature is significant**. All p-values are above 0.37 and every single-feature AUC lies between 0.49 and 0.52. In the generated rows every feature is significant ($p < 0.0001$), with glucose at 0.78 and the others between 0.55 and 0.62.

# 5 Results

## 5.1 Model comparison

The table shows cross-validated results from the training rows and results on the untouched test rows. All three models land very close together.

| Model | CV ROC-AUC | Test ROC-AUC | Accuracy | Precision | Recall | F1 |
|---|---|---|---|---|---|---|
| Logistic Regression | 0.816 ± 0.014 | 0.822 | 0.749 | 0.749 | 0.750 | 0.749 |
| Random Forest | 0.806 ± 0.015 | 0.816 | 0.743 | 0.741 | 0.748 | 0.744 |
| XGBoost | 0.810 ± 0.014 | 0.815 | 0.734 | 0.734 | 0.734 | 0.734 |
| Majority-class baseline | - | 0.500 | 0.500 | - | 0.000 | - |

**Logistic Regression was selected** because it had the highest cross-validated ROC-AUC (0.816). Its test ROC-AUC is 0.822 with a 95% bootstrap interval of 0.796 to 0.848. A paired bootstrap on the test rows (notebook 03) shows that the edge over Random Forest (0.006 AUC) is not clear and the edge over XGBoost (0.007) is very small. In practice the three models perform the same, so choosing the simplest and most explainable one is sound.

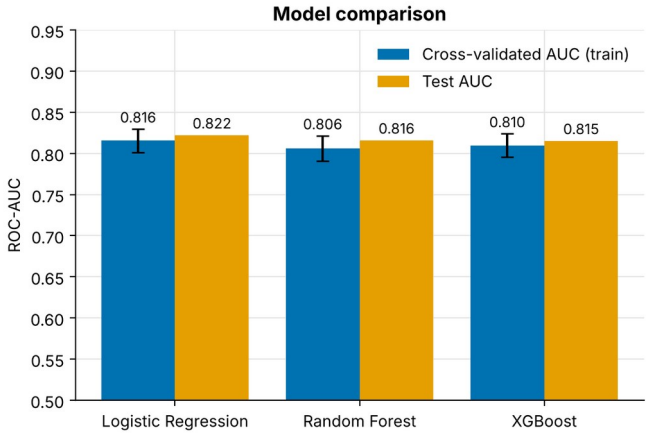



*Figure 7. Cross-validated and test ROC-AUC for the three models.*

## 5.2 ROC curves and calibration

The ROC curves lie almost on top of each other. The calibration plot asks whether a predicted risk of, say, 70% matches the real share of sick patients. All three models follow the diagonal closely, with small wobbles in the lowest and highest groups. The probabilities were not recalibrated.

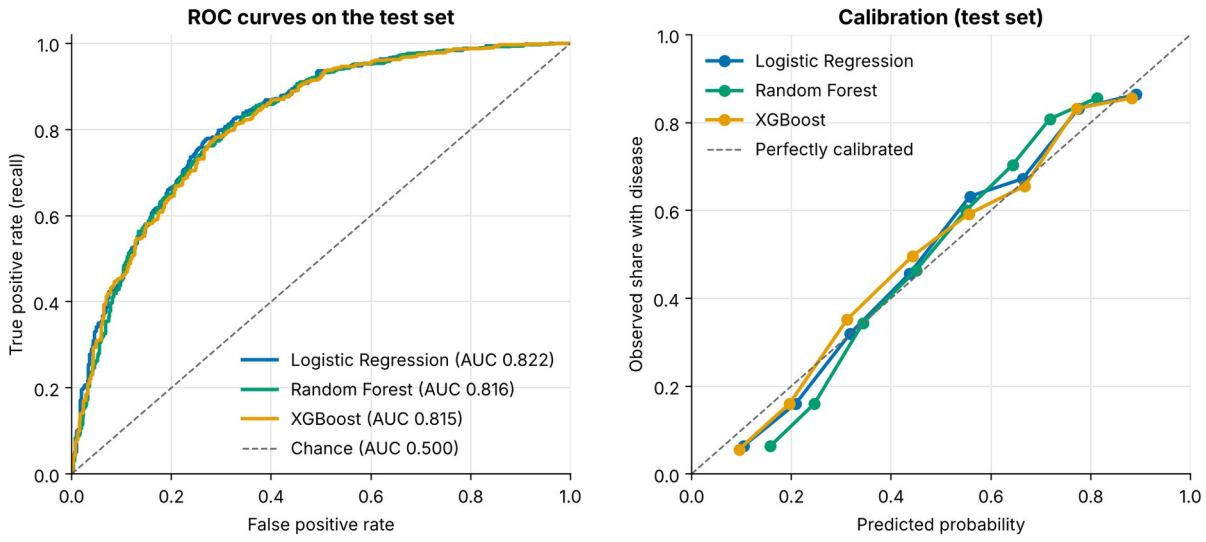



*Figure 8. ROC curves (left) and calibration curves (right) on the test set.*

## 5.3 Confusion matrices and the screening threshold

The first plot shows how precision and recall change with the cut-off, using out-of-fold training predictions. The dashed line marks the screening threshold.

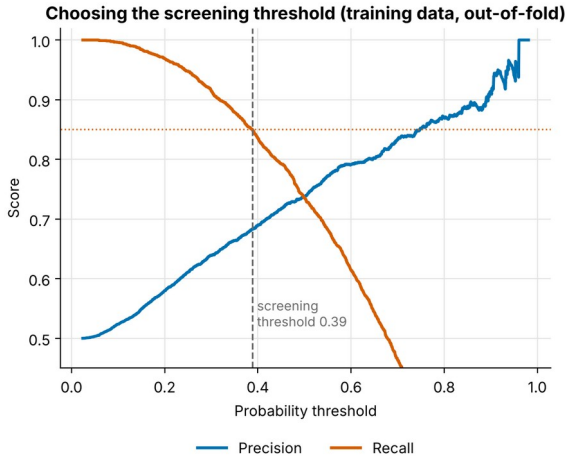



***Figure 9.*** *Precision and recall for every cut-off, with the chosen screening threshold.*

| Measure | Cut-off 0.50 | Screening cut-off 0.39 |
|---|---|---|
| Accuracy | 0.749 | 0.742 |
| Precision | 0.749 | 0.698 |
| Recall (sensitivity) | 0.750 | 0.854 |
| Specificity | 0.748 | 0.630 |
| F1 score | 0.749 | 0.768 |
| Missed sick patients (FN) | 125 of 500 | 73 of 500 |
| False alarms (FP) | 126 of 500 | 185 of 500 |

Moving to the screening cut-off raises recall from 0.75 to 0.85, so the number of missed patients falls from 125 to 73. The price is a lower specificity (0.75 to 0.63): 59 more healthy patients are flagged. A real programme would choose the cut-off together with doctors.

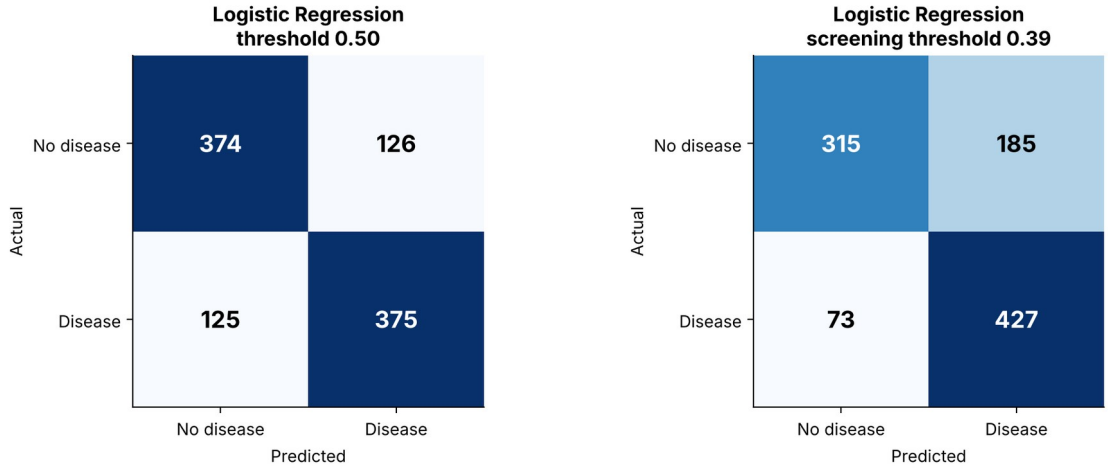



***Figure 10.*** *Confusion matrices of the selected model at cut-off 0.50 (left) and at the screening cut-off (right).*

## 5.4 Learning curve

Training and cross-validation scores meet at about 0.82 and stay flat as more rows are added. The model is not overfitting, and more rows of the same kind would not help much. The limit is set by the six features and the noise in the labels.

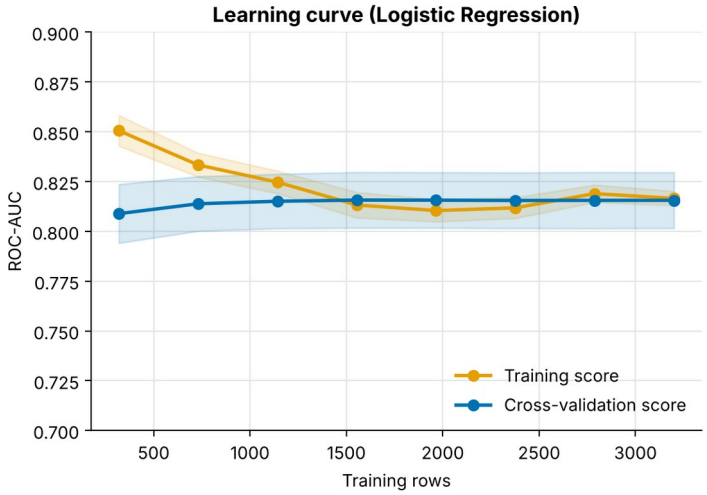



***Figure 11.*** *Learning curve of the selected model.*

## 5.5 Where does the predictive signal come from?

Models trained and cross-validated on the 500 original rows alone reach an AUC of only 0.45 (Logistic Regression) and 0.50 (Random Forest). The selected final model scores 0.50 on the 100 original test rows, but 0.86 on the 900 generated test rows.

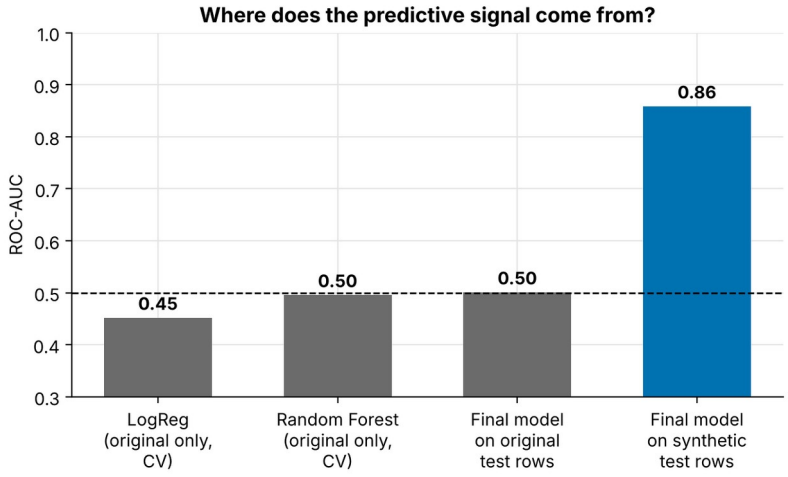



***Figure 12.*** *ROC-AUC on original versus generated rows.*

> **What this means**
>
> The headline AUC of 0.82 is an average of "no signal" (original rows) and "built-in signal" (generated rows). It shows that the pipeline can recover relationships that are present in the data. It does **not** show that the model would predict disease in real patients. This report therefore makes no clinical claim.

### 5.6 Subgroup performance

To check that the model works similarly for different kinds of patients, ROC-AUC and recall at the screening cut-off were computed for age bands, smokers and non-smokers, and BMI bands. ROC-AUC stays between 0.80 and 0.83 in all of them. Recall is higher for older patients (0.90 for ages 55 to 75, against 0.78 for ages 25 to 39), for smokers (0.88 against 0.82) and for BMI 30 or more (0.88 against 0.83), because these groups get higher scores overall. The price is more false alarms in those groups. Each group has 250 to 540 patients, so differences of a few points are within noise.

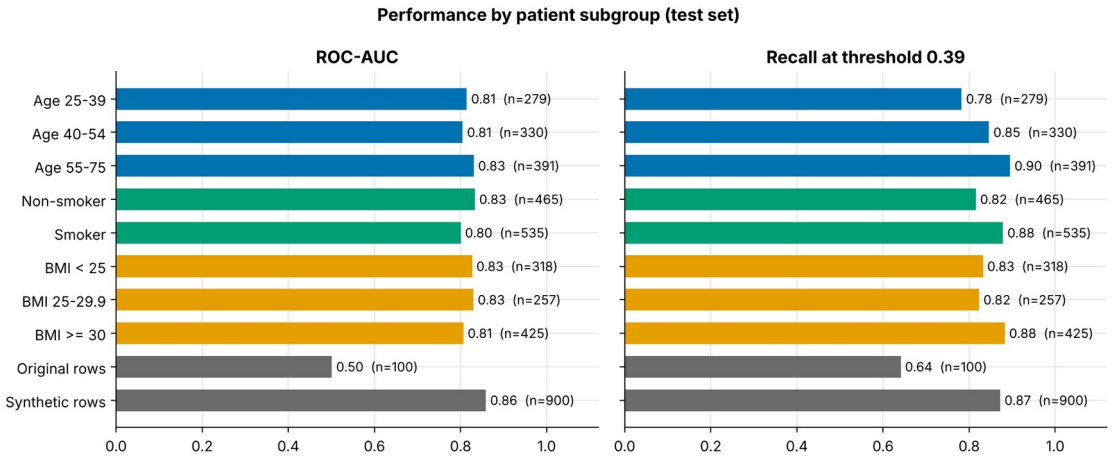



***Figure 13.*** *ROC-AUC and recall at the screening cut-off for patient subgroups (test set).*

## 6 Explainability with SHAP

### 6.1 What the selected model learned

Because the selected model is a Logistic Regression, its coefficients can be read directly. The table shows the effect of a one-standard-deviation increase in each feature, with all other features held fixed. An odds ratio of 3.0 means the odds of disease are three times higher.

| Feature | Effect per +1 SD (log-odds) | Odds ratio |
|---|---|---|
| Glucose | 1.09 | 2.99 |
| Blood pressure | 0.48 | 1.61 |
| Age | 0.37 | 1.45 |
| BMI | 0.37 | 1.45 |
| Cholesterol | 0.30 | 1.34 |
| Smoking | 0.25 | 1.29 |

### 6.2 Permutation importance

When glucose is shuffled on the test set, the AUC drops by 0.21. The next feature, blood pressure, causes a drop of only 0.04, followed by age (0.024), BMI (0.023), cholesterol (0.015) and smoking (0.013).

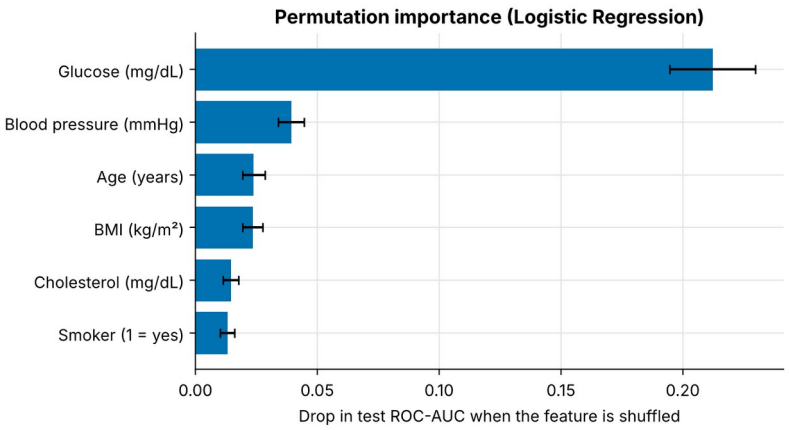



***Figure 14.*** *Drop in test ROC-AUC when each feature is shuffled.*

## 6.3 SHAP summary

Each dot is one patient. Dots to the right raised the predicted risk, dots to the left lowered it. Red means a high feature value and blue a low one. High glucose (red) sits far to the right, so high glucose raises risk strongly.

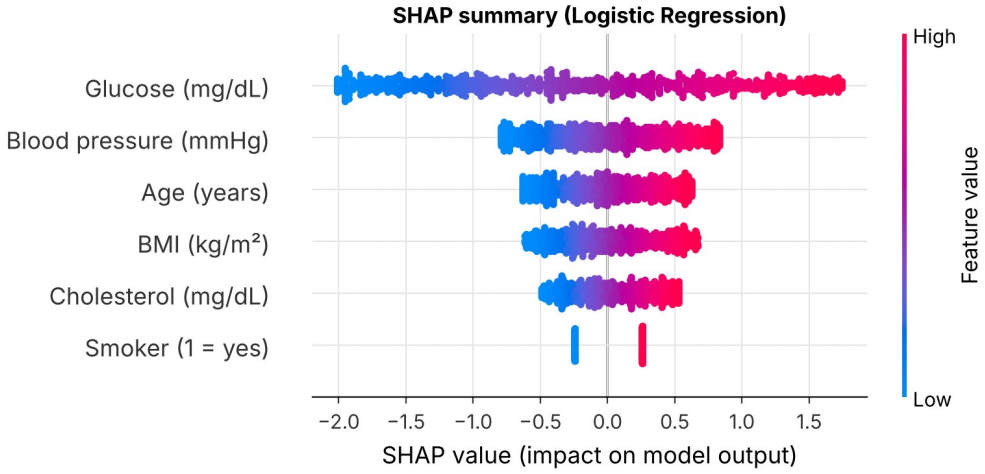



***Figure 15.*** *SHAP summary plot of the selected model (log-odds scale).*

All three models rank the features the same way. Glucose holds 38% to 47% of the total importance in each of them, which is reassuring: the finding does not depend on one algorithm.

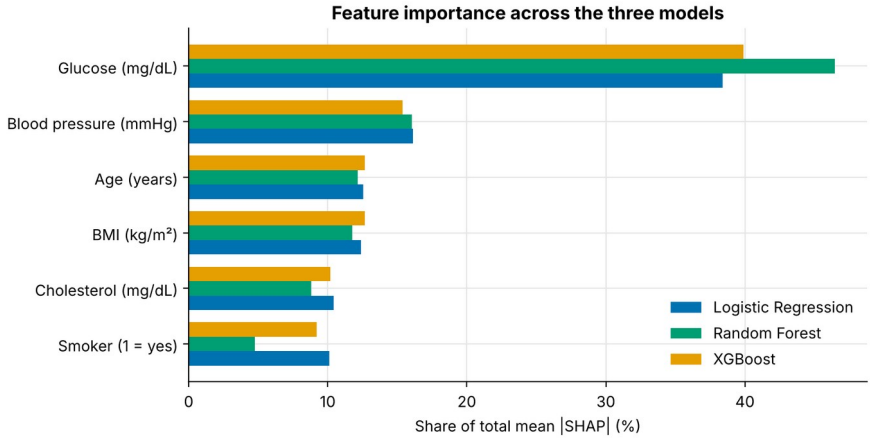



***Figure 16.*** *Share of total mean |SHAP| per feature for the three models.*

## 6.4 Shape of the glucose effect

The dependence plot for XGBoost shows glucose pushing the prediction up almost in a straight line, from about -2 for very low glucose to about +1.7 for very high glucose (log-odds). Because the real effect is nearly linear, a plain Logistic Regression can capture it, which explains why it matches the tree models.

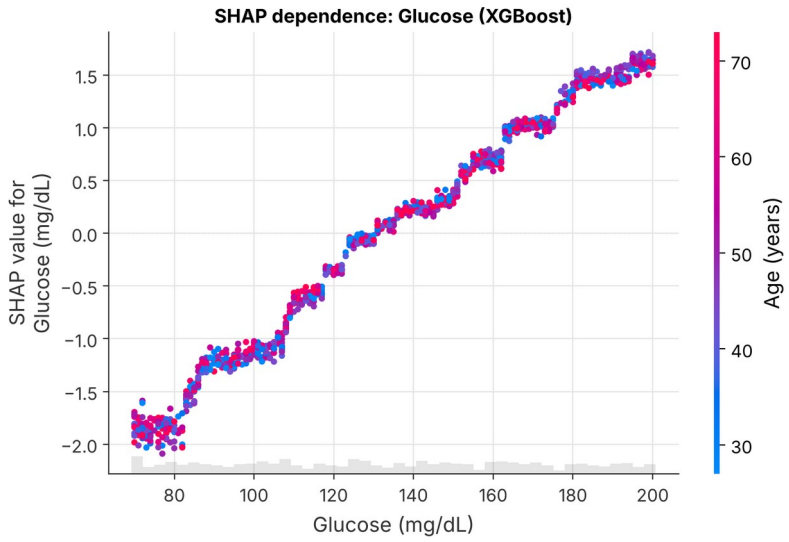



***Figure 17.*** *SHAP dependence plot for glucose (XGBoost), coloured by age.*

## 6.5 Explaining one patient

The highest-risk patient in the test set (id P4967) has glucose 197, blood pressure 136, cholesterol 271, age 65, BMI 34.1 and is a smoker. The selected model gives a risk of 98%. Glucose adds the most, then blood pressure and cholesterol. The Streamlit app shows the same kind of explanation in percentage points.

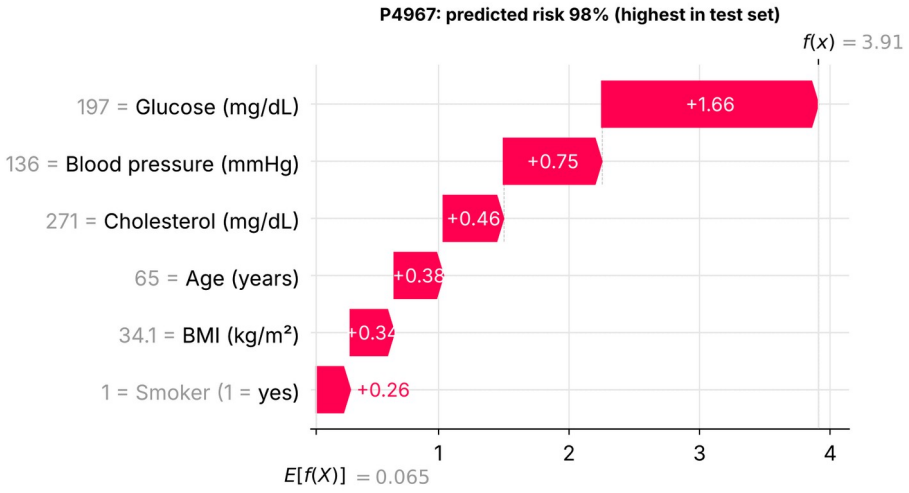



***Figure 18.*** *SHAP waterfall plot for the highest-risk patient in the test set.*

## 6.6 Who does the model miss?

At the screening cut-off the model misses 73 of the 500 sick test patients. Their average glucose is 107, close to the 116 of healthy patients and far below the 157 of the patients who were caught. The model

can only use the six columns it is given. Anything outside them, such as symptoms, family history or other lab tests, is invisible to it.

# 7 The Streamlit application

The prototype is in `app/streamlit_app.py` and starts with `streamlit run app/streamlit_app.py`. It loads the saved model and uses the same prediction and explanation functions as the notebooks (`src/predict.py`). A disclaimer is shown at the top of every page.

| Tab | What it shows |
|---|---|
| Risk check | Sliders for the six inputs (limited to the trained ranges). After pressing Estimate risk: the estimated probability, a risk band (lower, elevated or high), a bar chart of how each feature moved the estimate in percentage points, and a table of the same numbers. |
| Model performance | Headline metrics, the three-model comparison table, a warning about original versus generated rows, and the ROC, confusion-matrix and SHAP plots. |
| Data, ethics and limits | How the data was made, what the label means, and the ethical points and limits in plain words. |

Risk bands come from data, not from guesses: **lower risk** is below the screening threshold (0.39), **elevated risk** is between the screening threshold and 0.50, and **high risk** is 0.50 or above. Inputs are validated before use, so a value outside the trained range gives a clear error and not a silent wrong answer. Nothing typed into the app is saved to disk or sent anywhere.

If the saved model file cannot be loaded on another computer (for example because of a different scikit-learn version), the app refits the selected model from its saved settings. This takes a few seconds.

# 8 Testing and reproducibility

The project has 34 automated tests (run with `python -m pytest`). All of them pass.

| Test file | Tests | What it checks |
|---|---|---|
| `tests/test_data.py` | 13 | Shape, column order, sequential ids, first 500 rows identical to the original file, exact class balance, validation catches bad rows, cleaning removes them, stratified and reproducible split, data dictionary is filled in. |
| `tests/test_pipeline.py` | 8 | Scaler learns from training rows only (no leakage), every model fits and gives valid probabilities, model beats chance, metric formulas, the screening threshold reaches its target recall, bootstrap interval. |
| `tests/test_predict.py` | 13 | Saved model reproduces the reported test AUC, rebuilt model matches the saved one, prediction format, higher glucose gives higher risk, bad inputs are rejected, explanations add up exactly to the prediction. |

All random choices use seed 42. Running `python -m src.train` again gives the same numbers (the notebook run and the script run were compared). Training takes about two minutes on a 2-core machine. The exact library versions are in Appendix B.

# 9 Ethical considerations

> **Disclaimer**
> Disease prediction models must **not** be used for clinical diagnosis without medical supervision. This disclaimer appears in the app, in every notebook, in the README and in this report.

- **No real patients.** The data is an anonymous programme file plus generated rows. No personal health information was collected. Patient ids are codes.
- **Honest description of the data.** The data is called partly synthetic everywhere, and the finding that the original rows carry no signal is reported openly instead of being hidden behind one good-looking score.
- **Not a diagnosis.** The app gives a risk estimate for learning purposes. Its wording avoids telling a user that they do or do not have a disease.
- **Errors have a cost.** Missed patients get false comfort and false alarms cause worry and extra tests. The screening threshold makes this trade-off visible, and it should be set by clinicians in any real use.
- **Fairness checks.** Performance was compared across age, smoking and BMI groups. The groups are small and the data is synthetic, so the check is a first step and not proof of fairness. The data has no information on sex, ethnicity or income, so fairness for those groups could not be checked at all.
- **Transparency.** Every prediction can be explained with SHAP, and the code, settings and random seeds are included so the work can be inspected and repeated.
- **Privacy in the app.** Values typed into the app are used for the estimate and then dropped with the browser session. Streamlit usage statistics are turned off in `.streamlit/config.toml`.
- **AI assistance.** Parts of the code, the generated data and the documents were prepared with the help of an AI assistant (Claude, by Anthropic). The material was run, checked and reviewed, and the author takes responsibility for the submission.

# 10 Limitations

- **Partly synthetic data.** 90% of the rows are generated, and the feature-to-disease links in them were assumed. The reported scores measure how well the pipeline recovers those assumptions.
- **No signal in the original rows.** On the 500 original rows the models are no better than chance.
- **One combined label.** The file does not say which disease the label stands for.
- **Unclear units.** The type of blood pressure (systolic or diastolic) and the type of glucose test are not stated. Units for blood pressure, glucose and cholesterol were inferred from the value ranges.
- **Evenly spread features.** Real patients are not spread evenly across ranges, and real data has missing values, outliers and measurement errors that this file does not have.
- **One split, no outside validation.** Results come from one random split of one file. There is no external or prospective test.
- **Six features only.** No symptoms, family history, medicines, other lab tests, sex or lifestyle details.

- **No recalibration.** Probabilities are used as the models give them, although they look reasonably well calibrated.

# 11 Conclusion and future work

The project delivers what the brief asks for: a cleaned and documented dataset, an exploratory analysis with six visualisations, a fair comparison of Logistic Regression, Random Forest and XGBoost, SHAP explanations, a working Streamlit prototype and a report with its ethical points. The selected Logistic Regression reaches a test ROC-AUC of 0.822 and, at the screening cut-off, catches 85% of the sick test patients at a specificity of 0.63. Glucose is the dominant feature in every method used.

The most important lesson is about the data, not the models. The three algorithms are tied, and the original 500 records carry no learnable signal. A model is only as meaningful as the data behind it, so the honest conclusion is that the pipeline is ready and the next step is real data.

## Future work

- Replace the generated rows with real, properly approved clinical data (for example the Pima Indians Diabetes and Heart Disease datasets named in the brief) and repeat the study.
- Separate the label into diabetes, heart disease and hypertension, and add the missing features such as sex, family history and the type of blood pressure.
- Add external validation, probability recalibration and fairness checks for more groups.
- Package the model behind a REST API and add monitoring, if a real deployment is ever planned with clinicians.

# References

[1] UCI Machine Learning Repository, Diabetes dataset (AIM '94). https://archive.ics.uci.edu/dataset/34/diabetes. Link given in the project brief; it is not the Pima Indians data, so the mentor is to validate it.

[2] Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825-2830.

[3] Chen, T., and Guestrin, C. (2016). XGBoost: A scalable tree boosting system. Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 785-794.

[4] Breiman, L. (2001). Random forests. Machine Learning, 45(1), 5-32.

[5] Lundberg, S. M., and Lee, S.-I. (2017). A unified approach to interpreting model predictions. Advances in Neural Information Processing Systems 30.

[6] Streamlit documentation. https://docs.streamlit.io

[7] HCLTech Career Shaper Programme Guidelines (provided by the Programme Coordinator).

[8] Smith, J. W., Everhart, J. E., Dickson, W. C., Knowler, W. C., and Johannes, R. S. (1988). Using the ADAP learning algorithm to forecast the onset of diabetes mellitus. Proceedings of the Symposium on Computer Applications and Medical Care, 261-265. (Pima Indians Diabetes dataset, named in the dataset-generation prompt.)

# Appendix A  Project structure and how to run

```
early_disease_prediction/
  README.md                    overview and quick start
  requirements.txt             libraries (minimum versions)
  requirements-tested.txt      exact versions used for this report
  pytest.ini
  data/
    dataset.csv                the 500 original rows
    dataset_5000.csv           500 original + 4,500 generated rows
    data_dictionary.csv        all columns described
    processed/train.csv, test.csv   the exact split used
  src/                         reusable code
    config.py  data_utils.py  models.py  train.py
    evaluate.py  explain.py  predict.py  viz.py
    generate_synthetic_rows.py
  notebooks/
    01_EDA.ipynb  02_Model_Training.ipynb  03_Evaluation_and_SHAP.ipynb
  app/streamlit_app.py         the prototype
  models/                      saved models (.joblib)
  reports/                     metrics, figures, this report
  tests/                       34 automated tests
  docs/                        demo script, declaration, diagram code
```

**Quick start** (Python 3.10 or newer, from the project folder):

```
pip install -r requirements.txt
python -m src.train                     # optional: retrain (about 2 minutes)
python -m pytest                        # run the 34 tests
jupyter notebook notebooks/             # open the three notebooks
streamlit run app/streamlit_app.py      # start the app
```

# Appendix B  Environment

| Item | Version used |
|---|---|
| Python | 3.13.17 |
| scikit-learn | 1.9.1 |
| XGBoost | 3.4.1 |
| SHAP | 0.54.1 |
| pandas | 3.0.5 |
| NumPy | 2.5.3 |
| Random seed | 42 |
| Cross-validation | 5-fold, stratified, shuffled |
| Test share | 20% (1,000 rows) |